First, add the `G` parameter to the definition of the `update_params` function, and change `ret_ = G`:

```
def
update_params(worker_opt,values,logprobs,rewards,G,clc=0.1,gamma=0.
95):
    rewards = torch.Tensor(rewards).flip(dims=(0,)).view(-1)
    logprobs = torch.stack(logprobs).flip(dims=(0,)).view(-1)
    values = torch.stack(values).flip(dims=(0,)).view(-1)
    Returns = []
    ret_ = G
        ...
```

The rest of the function is exactly the same and is omitted here.

All we need to change in the `worker` function is to capture the newly returned `G` array and pass it to `update_params`:

```
def worker(t, worker_model, counter, params):
    worker_env = gym.make("CartPole-v1")
    worker_env.reset()
    worker_opt = optim.Adam(lr=1e-
4,params=worker_model.parameters())
    worker_opt.zero_grad()
    for i in range(params['epochs']):
        worker_opt.zero_grad()
        values, logprobs, rewards, G =
run_episode(worker_env,worker_model)
        actor_loss,critic_loss,eplen =
update_params(worker_opt,values,logprobs,rewards, G)
        counter.value = counter.value + 1
```

You can run the training algorithm again as before, and everything should work the same except with better performance. You might be surprised at how much more efficient *N*-step learning is. Figure 5.15 shows the plot of episode length over the first 45 seconds of training for this model.

Figure 5.15. Performance plot for distributed advantage actor-critic with true *N*-step bootstrapping. Compared to our previous Monte Carlo algorithm, the performance is much smoother due to the more stable critic.